



# FEATURE MODEL - PROBLEMATIC ARCHER MODEL DIAGNOSTIC VISUALIZATION

Bachelor's Project Thesis

Reinder Kas, s3189953, r.kas@student.rug.nl,  
Supervisor: Dr. H.A. de Weerd

**Abstract:** Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed at sodales magna, a tempus tellus. Sed ut dui imperdiet, pulvinar ligula nec, placerat orci. Nulla porttitor justo mi, vitae convallis nisl lacinia sit amet. Donec viverra pellentesque arcu, ut cursus mi commodo sit amet. Donec ut iaculis nibh, et feugiat enim. Ut sodales ipsum tellus, eu mattis sem consectetur vel.

## 1 Introduction

Within the field of Artificial Intelligence and subsequently Computer Science there are many complex real-world problems that researchers try to solve. Many of these problems consist of a set of Constraints given by the environment, or model, that the problem tries to emulate. These Constraints can be in many forms; from logical requirements to resource limitations (mostly used for optimization problems).

This paper will focus only on Constraint Satisfaction Problems that deal with the form of logical requirements called Feature Models.

### 1.1 Constraint Satisfaction Problem

A Constraint Satisfaction Problem is a mathematical problem that consists of three components  $X$ ,  $D$ , and  $C$ : (Russell & Norvig (2010))

$X$  is a set of variables,  $\{X_1, \dots, X_n\}$ .

$D$  is a set of domains,  $\{D_1, \dots, D_n\}$ .

$C$  is a set of constraints that specify allowable combinations of values.

If a problem is created with these three components then a Solver will be able to find a solution to the problem as long as one exists. Many of these solvers exist already, such as Microsoft Z3 Solver, Google OR-Tools, or choco-solver. Each of these solvers works in a different way but should ultimately come to the same solution if the

problem is defined correctly.

There can also be cases where a problem does not have a solution:

$$X = \{A, B\}.$$

$$D = \{A = \{True\}, B = \{False\}\}.$$

$$C = \{(A == B)\}$$

With this problem statement, no assignment in the Domains for Variables  $A$  and  $B$  will result in a satisfaction of the Constraints in the set  $C$ . In this case we can simply see from the Problem definition that the Constraint  $A == B$  gives a problem for the Features within the model. simply removing (or changing) this constraint is one solution. Another would be to expand each Variables' Domains such that they can be equal to one another.

When a model does not have a solution such as seen in the aforementioned model, it can be relatively easy for such small models to see what is wrong within a model's logic just by reasoning about the model definition (mostly about the Domain, and Constraint components). The bigger the model becomes, the more constraints the model will contain, the more complex a model's logic will be, and the more difficult it will be to pinpoint what constraint is the exact cause of the problem.

For this it would be ideal to have an interpreter or visualizer such that it is immediately clear where the problem within the model's logic lies.

## 1.2 Feature Model

(TODO: GIVEN IS THAT ROOT IS SELECTED)  
Within the field of Software Development there is a compact way to represent all the products of the Software Product Line, called a Feature Model (TODO: MORE INFO about SPL?).

This Feature Model is a visual way of modeling all the relationships (Constraints) and features (Variables) within a product, giving a overview of what is and is not possible within every aspect of a product.

Within this Feature Model, a Feature can be either selected (True) or not selected (False). Other values of the domain such as numeric values are also possible, but are beyond the scope of this research paper.

These Feature Models also have a predefined set of Constraints, divided into two relationship types: the Parent-Child relationships, and the Cross-Tree relationships (Benavides et al. (2010)).

The Parent-Child relationships define the structure and hierarchy (tree) of the model, defined as Mandatory, Optional, Or, and Alternative. The Cross-Tree relationships are relationships that are typically inclusion or exclusion statements. These make it possible to define relationships between Features throughout the whole hierarchy of the tree. The definition of these relationships between Parent (P) and Child (C) (A and B for Cross-Tree relationships) are given as follows: (Hotz et al. (2014))

| Constraint  | Semantics                                                                                                                                                                                                                                                            |
|-------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Mandatory   | $P \leftrightarrow C$                                                                                                                                                                                                                                                |
| Optional    | $P \rightarrow C$                                                                                                                                                                                                                                                    |
| Or          | $P \leftrightarrow (C_1 \vee C_2 \vee \dots \vee C_n)$                                                                                                                                                                                                               |
| Alternative | $(C_1 \leftrightarrow (\neg C_2 \wedge \dots \wedge \neg C_n \wedge P))$<br>$\wedge (C_2 \leftrightarrow (\neg C_1 \wedge \dots \wedge \neg C_n \wedge P))$<br>$\wedge \dots$<br>$\wedge (C_n \leftrightarrow (\neg C_1 \wedge \dots \wedge \neg C_{n-1} \wedge P))$ |
| Requires    | $A \rightarrow B$                                                                                                                                                                                                                                                    |
| Exclude     | $\neg A \vee \neg B$                                                                                                                                                                                                                                                 |

These definitions might be better understandable when translated to natural language (TODO: STILL NECESSARY?):

**Mandatory:** The Parent and Child have the same value (True or False).

**Optional:** The Child can only be selected iff the Parent is selected.

**Or:** There must be at least one Child selected iff the Parent is selected.

**Alternative:** There must be exactly one Child selected iff the Parent is selected.

**Requires:** If Feature A is selected, then Feature B is selected as well

**Excludes:** Both Features A and B cannot be selected at the same time

One example that is often used to showcase the usage of these Constraints is the Mobile Phone Feature Model as shown below in Figure 1.1.

This model gives a compact and intuitive representation of a Mobile Phone such that the underlying relationships between all the Features within the Feature Model are visible in a visual representation.

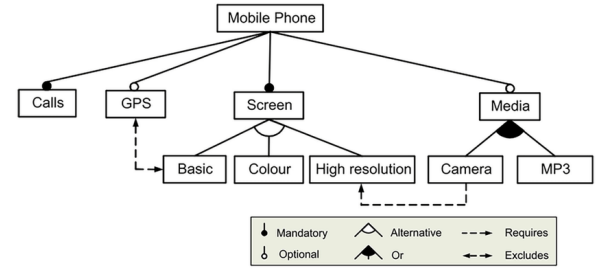


Figure 1.1: Feature Model representation of a simple phone (Benavides et al. (2010)).

## 1.3 Elfsquad

(TODO: DOES THIS BELONG IN A RESEARCH PAPER?)

Elfsquad is a company that specializes in online Product Configurations. The basic principles that satisfy most requirements for their customers consist of Constraints that are only logical Constraints. After having developed their own software that allows customers to create their own Feature Models, it became apparent that these models become complex very quickly.

The average Feature Model within the Elfsquad software consists of (TODO: CHECK AVERAGE VALUES!) 1000 Features and 1500 Constraints (upwards to 12.000 Features and 25.000 Constraints). Their customers often have problems with their Feature Models, where these models do not have a solution, meaning their product cannot be configured. This is a massive problem for customers, because not being able to configure a product means a loss of revenue with wasted time that could have been spent selling a product. In a lot of these cases the customers call Elfsquad support to troubleshoot their problems, which in turn now also costs Elfsquad extra resources that could and should have been avoided in the first place.

The solution to this problem is to find a method to indicate to the customer what part of the model causes the Feature Model to not have a solution. An easy method of doing so would be a visualization tool that shows the problem within the model. Such visualization tools already exist for Feature Models (FeatureIDE) but this visualization tool does not give enough functionality for the Elfsquad company to integrate with.

Elfsquad has created their own method of creating Feature Models through their in-house developed Archer Modeling Language. This Archer Modeling Language is more expansive than the well-known Feature Model Constraints that FeatureIDE relies on to function.

This means that a different solution must be found. Since Elfsquad has developed their own Modeling Language, it will be a difficult task to find an already existing program that flawlessly integrates with their language. The best solution for this problem is to create this visualization tool from scratch.

## 1.4 Archer Modeling Language

Archer is the in-house developed Modeling Language of the company Elfsquad. This language can be seen as a black box that takes text (in String format) as input, and as outputs a Microsoft Z3 Solver object containing all the problem definitions as given in the input text. Further specifications and implementations of this Archer Modeling Language cannot be disclosed as per a NDA signed with the company.

Since this language is a modeling language for Feature Models, we can model the Mobile Phone example as seen in Figure 1.1 as an Archer model as well:

```
model [Phone] {
    [Phone] - mandatory -> [Calls]
    [Phone] - optional -> [GPS]

    [Phone] - mandatory -> [Screen]
    [Screen] - alternative -> [Basic]
    [Screen] - alternative -> [Color]
    [Screen] - alternative -> [High Res]

    [Phone] - optional -> [Media]
    [Media] - or -> [Camera]
    [Media] - or -> [MP3]

    [Camera] - requires -> [High Res]
    [GPS] - excludes -> [Basic]
}
```

It is also easy to create a small model for which there is no solution:

```
model [root] {
    [root] - mandatory -> [child]
    [child] - excludes -> [child]
}
```

Again, it is clear that there will be no solution since the `child` Feature must be selected because of the (`root - mandatory → child`) Constraint, but at the same time cannot be selected due to the (`child - excludes → child`) Constraint, thus there is no solution.

## 2 Problem Description

With the development of Archer Language there is no visual representation of what the logic means within a model. Even smaller models could become problematic to diagnose since all information pertaining to the model is written in a textual format.

As of right now, Elfsquad's support spends too much time diagnosing Feature Models that do not have a solution. The main problem here is that these models that do not have a solution should not have been created in the first place. If they are

created by customers, then there should exist a tool that empowers the customer to self-diagnose the problem within their own Feature Model. If this tool is easy enough to use and shows the correct amount of information, then the amount of time for the Elfsquad support will spend diagnosing Feature Models should greatly reduce, giving them more time to help with different matters.

## 3 Method

### 3.1 Z3

In order to solve the problem for Elfsquad, this thesis is about creating a visualization tool for the Archer Modeling Language.

Since the Archer Modeling Language makes use of the Microsoft Z3 Solver it is best to use the Z3 solver for this diagnostics tool as well.

Upon initializing a new Z3 solver object, it is possible to set flags that determine the behaviour of the Z3 Solver. The two flags most relevant for what this tool wants to do are the flags **proof**, and **unsat\_core**.

The **proof** flag tells the Z3 Solver to generate a proof in Lisp showing the internal logical structure of the Z3 Solver as to why it could not satisfy the given Constraints.

The **unsat\_core** flag tells the Z3 Solver to generate the minimal set of Constraints that is still unsatisfiable (Unsatisfiable Core). This tool will produce a visual representation of the entire given Feature Model and highlight the Constraints that are present within the Unsatisfiable Core. This shows the end-user where the problem in the model is according to the Z3 Solver in a graphical way.

### 3.2 Experiment

In order to test the effectiveness of this tool, 10 Elfsquad employees will have to diagnose 10 predefined problematic Archer Models.

The models are increasing in size, starting with a small model and increasing to the biggest model. Each employee will have to diagnose the problem within all 10 models.

There are two methods of diagnosing this problem. One method would be to use the Archer Mod-

eling Language model definition, and the other method will be to use the graph visualization tool that is created for the Archer Modeling Language. Each employee's method of diagnosing the model will be randomized such that they have 5 Archer Language models, and 5 graphical visualized models. All models will also have 5 employees diagnosing the model as Archer representation and 5 as graph representation to make sure all model results can be compared to each other correctly.

This method allows for comparison between the time it takes the Elfsquad employees to diagnose the problem within each model in the Archer representation and the graphical representation.

## 4 Design Choises

## 5 Results

(TODO: GRAPH/TABLE OF RESULTS)

## 6 Conclusion

This thesis is trying to find a method to reduce the time spent on diagnosing problems within a Feature Model that does not have a solution. A visualization tool has been created to reduce this time and has to

The data clearly shows that with the use of the visualization tool the amount of time spent on diagnosing the problem within a Feature Model is greatly reduced.

## 7 Critique

With this thesis the visualization tool has only been tested and used by employees of Elfsquad. The ultimate goal is to have the customers of Elfsquad to use the tool in stead of the employees. This is not possible yet, as the Archer Modeling Language itself is still in development and set to be released later on.

The customers are scheduled to receive access to this tool at the same time as access to the Archer Modeling Language.

Furthermore, as of right now the newest version of the Archer Modeling Language has not been implemented into the tool yet.

The Archer Language is an ongoing project developed in-house at Elfsquad. Keeping up to date with the newest version of Archer would be too much implementation jumping from version to version that the decision has been made to take an older version of Archer and implement this into the tool.

Once Archer is completed, the tool is operational, and the tool has shown the company that the tool is able to increase the operational workflow for Elfsquad, then the complete version of Archer can be implemented into the tool for future use.

Another point of critique could

## 8 Future Research

This thesis functions as the basis as a diagnostics tool for the Elfsquad Archer Modeling Language. This means that the future research depends on the useage of the Archer Modeling Language and the ways that Elfsquad's customers want to use the tool. Since Archer is still in development within Elfsquad, there is no saying exactly what the customers will come up with how they want to use it. This however does not mean that no future research can be thought of:

### **Complete Archer Modeling Language implementation.**

For now there is only a limited component of the Archer Modeling Language implemented in the tool. Ongoing development into Archer has prevented this tool from keeping up with the latest version, as it would be too much work to keep up with all and every change as they come.

#### **8.0.1 Generate solutions to the problem.**

In stead of only showing where the problem is in the model, it is better if the tool gives suggestions as to how to fix the model to make it functional again. This generation of solutions can be done in a multiple of ways but is planned for further development of this tool.

### **Implement Numeric Domains.**

This tool currently only works with Features that make use of Boolean Domains. A Feature can be selected or not. Elfsquad has encountered Feature Models where customers want to make use of Numeric Domains as well. These Numeric Domains are acted upon by different Constraints, ones that are not present in the simple Boolean Feature Models that are discussed in this paper.

In order to support these Constraints, new visualizations should be added to the tool in order to give information pertaining to the numeric Constraints in the Model.

### **Interpret the Z3 generated proof.**

As of right now nothing is being done with the Z3 generated proof. This proof does contain all the information as to why it is not possible to find a solution and perhaps an easier interpretation can be created for diagnosing problematic Feature Models using this proof.

## References

- Benavides, D., Segura, S., & Ruiz-Cortés, A. (2010, 09). Automated analysis of feature models 20 years later: A literature review. *Information Systems*, 35, 615-636. doi: 10.1016/j.is.2010.01.001
- Hotz, L., Felfernig, A., Stumptner, M., Ryabokon, A., O'Neill, C., & Wolter, K. (2014, 04). Chapter 6. configuration knowledge representation and reasoning. In (p. 41-72). doi: 10.1016/B978-0-12-415817-7.00006-2
- Russell, S. J., & Norvig, P. (2010). *Artificial intelligence: A modern approach*. Prentice Hall.